

Algebra numerica

Nicolò Giuliani

Indice

1	Zeri di funzione	2
1.1	Metodo di bisezione	2
1.2	Metodo di punto fisso	2
1.3	Metodo di Newton	3
2	Ottimizzazione	5
2.1	Metodo del gradiente	5
2.2	Metodo di Newton puro	6
2.3	Gradiente stocastico (SGD)	7
3	Approssimazione dei dati	8
4	Algebra lineare numerica	10
4.1	Norme vettoriali e matrici	10
5	Sistemi lineari	10
5.1	Fattorizzazione Cholesky	11
5.2	Fattorizzazione LU	11
5.3	Condizionamento	15
5.4	Problema minimi quadrati e SVD	16
6	Imaging e problemi inversi	22
6.1	Compressione SVD	22
6.2	Deblur	22

1 Zeri di funzione

1.1 Metodo di bisezione

Dato un $a, b \in D$ tale che $f(a) < 0, f(b) > 0$:

Prendiamo in considerazione gli intervalli

$$I_k = [a_k, b_k]$$
$$I_k \subset I_{k+1} \subset \dots \subset I_1$$

con $f(a_k)f(b_k) < 0$:

$$f(c_k) = f\left(\frac{a_k + b_k}{2}\right)$$

Tale che

$$I_{k+1} = [a_{k+1}, b_{k+1}] = \begin{cases} [a_k, c_k] & \text{se } f(c_k) > 0 \\ [c_k, a_k] & \text{se } f(c_k) < 0 \end{cases}$$

Fissato δ come tolleranza di errore possiamo calcolare il numero di iterazioni k

$$k \geq \log_2 \frac{b-a}{\delta}$$

o per calcolare l'errore assoluto al passo k :

$$E_a = |x_k - x^*|$$

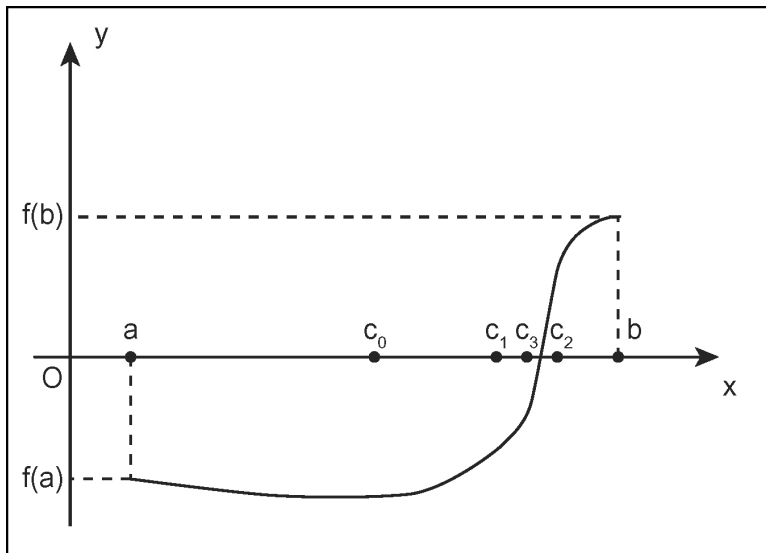


Figura 1: Esempio di esecuzione del metodo di punto fisso

1.2 Metodo di punto fisso

Per cercare $x^* \mid f(x^*) = 0$ possiamo cercare invece $x^* \mid g(x^*) = x^*$ tale che

$$g(x) = x - f(x)\phi(x)$$
$$|g'(x)| < 1 \text{ vicino ad } x^*$$
$$\rightarrow \text{converge ad } x^*$$

Iteriamo:

$$x_{k+1} = g(x_k)$$

$$\Rightarrow \boxed{x_{k+1} = x_k - f(x)\phi(x)}$$

Possiamo approssimare con Taylor $x_0 = x^*$

$$g(x) = \sum_{n=0}^{\infty} \frac{g^{(n)}(x_0)}{n!} (x - x_0)^n$$

$$\approx g(x^*) + g'(x^*)(x - x^*) + \frac{g''(x^*)}{2!} (x - x^*)^2 + \dots$$

L'errore al passo k lo definiamo come $e_k := |x_k - x^*|$

$$\begin{aligned} e_{k+1} &= |g(x_k) - x^*| \\ &= |g(x_k) - g(x^*)| \\ &= |g(x^*) + g'(x^*)(x_k - x^*) + \frac{g''(x^*)}{2!} (x_k - x^*)^2 + \dots - g(x^*)| \\ &= |g'(x^*)(x_k - x^*) + \frac{g''(x^*)}{2!} (x_k - x^*)^2 + \dots| \\ &= |g'(x^*)e_k + \frac{g''(x^*)}{2!} e_k^2 + \dots| \end{aligned}$$

Quindi approssimando possiamo dire

$$e_{k+1} \approx |g'(x^*)e_k + o(e_k)|$$

L'errore si riduce proporzionalmente a $e_k \rightarrow$ **convergenza lineare**.

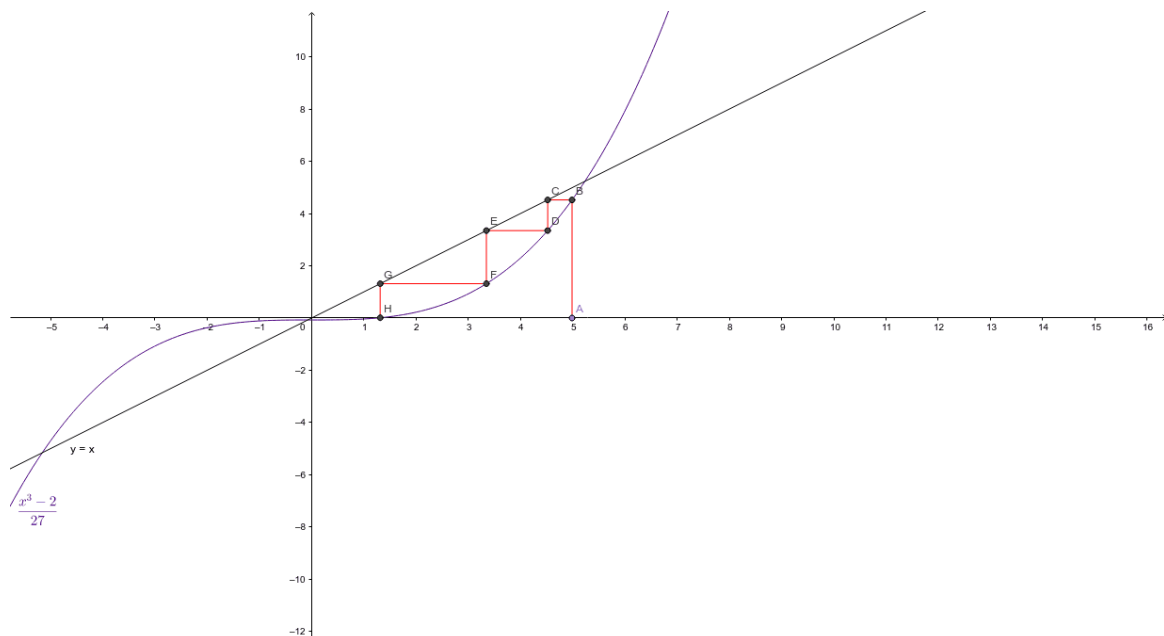


Figura 2: Esempio di esecuzione del metodo di bisezione

1.3 Metodo di Newton

Ora vediamo un caso speciale del punto fisso, se nell'algoritmo precedente abbiamo detto che

$$e_{k+1} \approx |g'(x^*)e_k + \frac{g''(x^*)}{2!} (e_k)^2|$$

se

$$\begin{aligned} 0 < |g'(x)| < 1 \\ e_{k+1} &\approx |g'(x^*)e_k + o(e_k)| \\ &\rightarrow \text{Convergenza lineare} \end{aligned}$$

Quindi se prendiamo

$$\begin{aligned} |g'(x)| &= 0 \\ e_{k+1} &\approx \left| \frac{g''(x^*)}{2!} (e_k)^2 + o(e_k^2) \right| \\ e_{k+1} &= C \cdot |e_k|^2 \quad \text{con } C > 0 \\ &\rightarrow \textbf{Convergenza quadratica} \end{aligned}$$

Per fare quello dobbiamo fare

$$\begin{aligned} g'(x) = 0 &= \frac{d}{dx}[x - f(x)\phi(x)] \\ &= 1 - f'(x)\phi(x) + \phi'(x)f(x) \end{aligned}$$

e dato che stiamo cercando $x^* \mid f(x^*) = 0$:

$$g'(x^*) = 1 - f'(x)\phi(x)$$

e dato che $g'(x^*) = 0$

$$\begin{aligned} 1 - f'(x)\phi(x) &= 0 \\ \rightarrow \phi(x) &= \frac{1}{f'(x)} \end{aligned}$$

Ricapitolando possiamo cercare

$$g(x) = x - \frac{f(x)}{f'(x)}$$

Ovvero algebricamente possiamo iterare:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

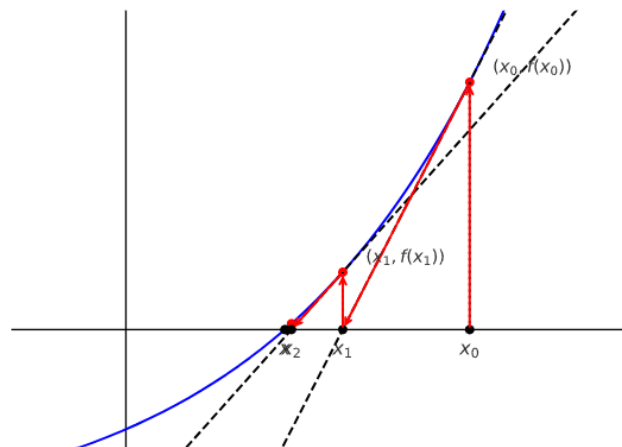


Figura 3: Esempio di esecuzione del metodo di Newton

2 Ottimizzazione

Vogliamo trovare $x^* = \arg \min f(x)$.

2.1 Metodo del gradiente

Partendo dal metodo del punto fisso possiamo definire che vogliamo cercare $x \mid \nabla f(x) = 0$:

$$x_{k+1} = x_k - f(x_k)\phi(x_k)$$

Definiamo per il punto fisso $f(x) = \nabla f(x_k)$:

$$\Rightarrow x_{k+1} = x_k - \nabla f(x_k)\phi(x_k)$$

Dato che $\nabla f(x)$ indica la direzione di massima crescita noi prendiamo come direzione di ricerca la direzione di decrescita massima:

$$p_k = -\nabla f(x_k)$$

tale che

$$\begin{aligned} p_k^T \nabla f(x_k) &< 0 \\ \Rightarrow -\nabla f(x_k)^T \nabla f(x_k) &< 0 \\ \Rightarrow \boxed{-\|\nabla f(x_k)\|_2^2} &< 0 \end{aligned}$$

Quindi avremo

$$x_{k+1} = x_k + p_k \phi(x_k)$$

Definiamo la funzione $\phi(x_k) = \alpha_k \in \mathbb{R}$.

Iteriamo con $\alpha_k > 0$ (*step size*)

$$\boxed{x_{k+1} = x_k - \alpha_k \nabla f(x_k)}$$

Dobbiamo stare attenti che α_k ci permetta di discendere per la funzione:

$$f(x_{k+1}) < f(x_k)$$

Come scegliamo α_k ?

- $\forall k, \alpha_k = n \in R$ (statico)
- **Line search**
 - Condizione di Armijo (costante di diminuzione del passo: $0 < c < 1$):
garantisce che il passo α faccia diminuire sufficientemente la funzione

$$\begin{aligned} f(x_{k+1}) &\leq f(x_k) - c\alpha(\nabla f(x_k))^T(\nabla f(x_k)) \\ f(x_{k+1}) &\leq f(x_k) - \underbrace{c\alpha\|\nabla f(x_k)\|^2}_{\substack{\text{diminuzione} \\ \text{minima accettabile}}} \end{aligned}$$

- Tecnica di backtracking:
dato un valore $\alpha = 1$ iniziale, riduciamo questo valore finche non troviamo un α_k che soddisfi condizione di Armijo.

Algorithm 1 Tecnica di backtracking

```

1: Input:  $x_k, f(), \nabla f(), \alpha_0 > 0, c \in (0, 1), p \in (0, 1)$ 
2:  $\alpha = \alpha_0$ 
3: repeat
4:    $x_{k+1} = x_k - \alpha \nabla f(x_k)$ 
5:   if  $f(x_{k+1}) > f(x_k) - c\alpha \|\nabla f(x_k)\|^2$  then
6:      $\alpha = p\alpha$ 
7:   end if
8: until  $f(x_{k+1}) \leq f(x_k) - c\alpha \|\nabla f(x_k)\|^2$ 
9: return  $\alpha$ 

```

Quindi nelle realta' implementiamo:

Algorithm 2 Metodo del gradiente

```

1: Input:  $x_0, f(), \nabla f(), \alpha_0 > 0, c \in (0, 1), p \in (0, 1)$ 
2:  $x_k = x_0$ 
3: for ( $k = 0; k < \text{max iter}; k++$ ) do
4:    $\alpha = \text{backtracking}(x_k, f(), \nabla f(), \alpha_0, c, p)$ 
5:    $x_{k+1} = x_k - \alpha \nabla f(x_k)$ 
6: end for
7: return  $x_{k+1}$ 

```

2.2 Metodo di Newton puro

Dal metodo del gradiente sappiamo (a sua volta basato sul punto fisso):

$$x_{k+1} = x_k - \nabla f(x_k) \phi(x_k)$$

Prendiamo $\phi(x) = H_f(x_k)^{-1}$

$$x_{k+1} = x_k - \nabla f(x_k) H_f(x_k)^{-1}$$

con $H_f(x_k)$ matrice Hessiana.

Quindi non iteriamo piu' con un passo statico o backtracking ma la hessiana misura la curvatura della funzione in tutte le direzioni in modo tale da modulare il passo:

- Direzioni molto "ripide" $\rightarrow$ passo più piccolo
- Direzioni piatte $\rightarrow$ passo più lungo

$$x_{k+1} = x_k - H_f(x_k)^{-1} \nabla f(x_k)$$

La funzione converge (quadraticamente) solo se $H_f(x_k)$ e':

- Simmetrica: $A = A^T$
- **Definita positiva:** $\forall v \neq 0 \in \mathbb{R}^n : v^T A v > 0$ o anche possiamo vedere se $\forall \lambda_A > 0$.

Questo metodo cosi come e' non e' utilizzato perche' "troppo rognosa", esistono pero' delle varianti molto efficaci.

2.3 Gradiente stocastico (SGD)

Dato una funzione del tipo

$$f(x) = \sum_{i=1}^N f_i(x)$$

e vogliamo trovare $x^* = \arg \min f(x)$, avremo dunque bisogno del suo gradiente. Che puo' essere scritto anche come

$$\nabla f(x) = \sum_{i=1}^N \nabla f_i(x)$$

Con il SGD andiamo ad approssimare il gradiente originale con solo un set di dati della funzione completa.

$$\nabla f(x) \approx \sum_{i \in \{i_1, \dots, i_m\}} \nabla f_i(x)$$

Fissato m scelgo il **mini batch** $i_1, \dots, i_m$, o se $m = 1$ il **batch**, casualmente **senza ripetizione** in $[1, N]$.

Quindi potremo iterare nel metodo del gradiente

$$\begin{aligned} x_{k+1} &= x_k - \alpha_k \nabla f(x_k) \\ x_{k+1} &\approx x_k - \alpha_k \left(\sum_{i \in \{i_1, \dots, i_m\}} \nabla f_i(x_k) \right) \\ &= x_k - \alpha_k (\nabla f_{i_1}(x_k) + \dots + \nabla f_{i_m}(x_k)) \end{aligned}$$

Epoca: Numero di iterazioni necessarie per aver utilizzato tutti gli N dati del dataset almeno una volta.

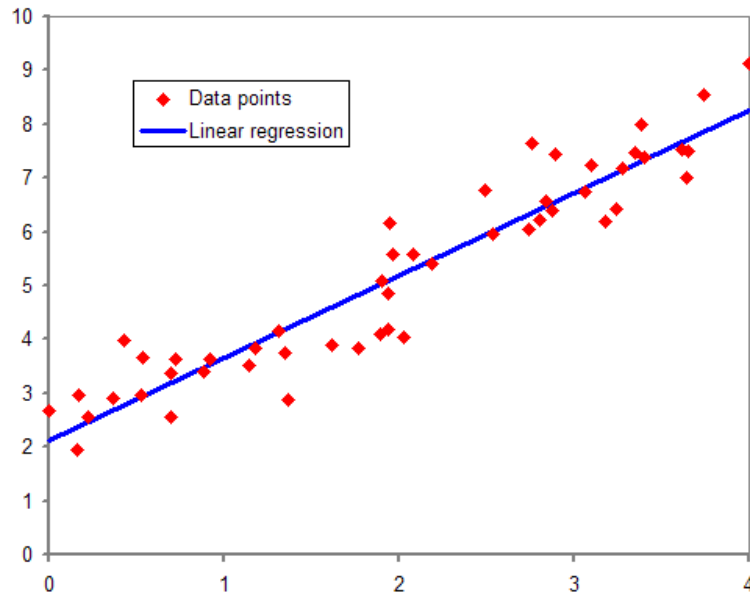
$$p = \frac{N}{m}$$

Ovvero dopo p iterazioni (usando p mini-batch da m dati ciascuno) tutti i dati da 1 a N sono stati utilizzati almeno 1 volta.

Questo e' utile in IA per monitorare l'apprendimento (se dopo p epoche il modello e' ancora stupido proviamo con $p + 1$ epoche).

3 Approssimazione dei dati

Dato dei dati $\theta = (x_i, y_i)$ dobbiamo cerca una funzione coerente $f(\bar{x}; \theta)$ con $\bar{x} \in [\min(x), \max(x)]$.



Per stabilire se una funzione sia piu' corente rispetto ad un altra dobbiamo avere una unita' di misura:

Definiamo la **loss function**

$$\mathcal{L}(\theta) = \sum_{i=1}^n (f(x_i; \theta) - y_i)^2$$

Ovvero dobbiamo trovare

$$\min_{\theta} \mathcal{L}(\theta)$$

La funzione basata sulla minimizzazione della perdita si chiama **regressione**.

- $k = 1$: $f(x; \theta) = \theta_0 + \theta_1 x$ (pol. grado 1)
- $k = 2$: $f(x; \theta) = \theta_0 + \theta_1 x + \theta_2 x^2$ (pol. grado 2)
- k : $f(x; \theta) = \theta_0 + \theta_1 x + \theta_2 x^2 + \dots + \theta_k x^k$ (pol. grado k)

Quindi la loss function per la regressione generale e':

$$\mathcal{L}(\theta) = \sum_{i=1}^n (\theta_0 + \theta_1 x + \theta_2 x^2 + \dots + \theta_k x^k - y_i)^2$$

Possiamo definire quindi:

$$\bullet \theta = \begin{pmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_k \end{pmatrix}$$

- $y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$

- Matrice $n \times k + 1$ come: $\Phi = \begin{pmatrix} x_1^0 & x_1^1 & \dots & x_1^k \\ x_2^0 & x_2^1 & \dots & x_2^k \\ \dots & \dots & \dots & \dots \\ x_n^0 & x_n^1 & \dots & x_n^k \end{pmatrix}$

Allora definiamo la loss function:

$$\mathcal{L} = \|\Phi\theta - y\|_2^2$$

E per ottenere θ facciamo:

$$\min_{\theta \in \mathbb{R}^{k+1}} \|\Phi\theta - y\|_2^2$$

Questo e' chiamato **problema dei minimi quadrati (lineare)**.

4 Algebra lineare numerica

4.1 Norme vettoriali e matrici

La distanza tra due vettori $x, y \in \mathbb{R}^n$ si calcola con:

- Norma di Manhattan: $\|x - y\|_1 = \sum_{i=1}^n |x_i - y_i|$
- Norma euclidea: $\|x - y\|_2 = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$
- Norma del massimo: $\|x - y\|_\infty = \max_{1 \leq i \leq n} |x_i - y_i|$

Vediamo norme di matrice:

- Norma 1 (matrice) : $\|A\|_1 = \max_j \sum_i |a_{ij}|$
- Norma 2 (matrice) : $\|A\|_2 = \sigma_{\max}(A)$
- Norma Frobenius : $\|A\|_F = \sqrt{\sum_{i,j} |a_{ij}|^2}$

5 Sistemi lineari

Definiamo A invertibile se e solo se $\dim(A) = rk(A)$.

A invertibile = A non singolare.

A non invertibile = A singolare.

Dato

$$Ax = b \quad A \in \mathbb{R}^{n \times n}$$

- Il sistema ammette 1 sola soluzione $\forall b \in \mathbb{R}^n$ sse A non e' singolare (= A invertibile)
 $\iff \det(A) \neq 0 \iff \exists A^{-1}$

$$Ax = b$$

$$A^{-1}Ax = A^{-1}b$$

$$\boxed{x = A^{-1}b}$$

Questo vale solo se A quadrata e invertibile.

Nota: Computazionalmente non si calcola $x = A^{-1}b$ perche' $O(n^3)$.

Se A non e' quadrata facciamo **CGLS** (Conjugate Gradient):

$$A^T Ax = A^T b$$

Così avremo la matrice $A^T A$ quadrata e definita positiva (SPD).

- Algoritmi diretti per la soluzione:

5.1 Fattorizzazione Cholesky

Per A simmetrica ($A = A^T$) e definita positiva:

$$A = LL^T$$

con

- L matrice singolare inferiore
- L^T la sua trasposta

$$\begin{cases} Ly = b \\ L^T x = y \end{cases}$$

Costo computazionale: $O(\frac{1}{n^3})$.

5.2 Fattorizzazione LU

Per A quadrata:

Dando per scontato che definiamo:

$$\underbrace{\begin{bmatrix} x & x & x & x \\ 0 & x & x & x \\ 0 & 0 & x & x \\ 0 & 0 & 0 & x \end{bmatrix}}_{\text{matrice triangolare superiore}} \quad \begin{cases} x_n = \frac{y_n}{t_{n,n}} \\ x_{n-1} = \frac{y_{n-1} - t_{n-1,n}x_n}{t_{n-1,n-1}} \\ x_{n-2} = \frac{y_{n-2} - t_{n-2,n-1}x_{n-1} - t_{n-2,n}x_n}{t_{n-2,n-2}} \\ \vdots \\ x_1 = \dots \end{cases}$$

$$\underbrace{\begin{bmatrix} x & 0 & 0 & 0 \\ x & x & 0 & 0 \\ x & x & x & 0 \\ x & x & x & x \end{bmatrix}}_{\text{matrice triangolare inferiore}} \quad x_i = \frac{y_i - \sum_{j=1}^{i-1} s_{ij} \cdot x_j}{s_{ii}}$$

Per calcolarle (ognuna): $O\left(\frac{n(n+1)}{2}\right) = O\left(\frac{n^2}{2}\right)$

Possiamo fattorizzare A :

$$A = L \cdot U$$

- L matrice triangolare inferiore con elementi uguale a 1 sulla diagonale
- U matrice triangolare superiore

$$\begin{aligned} Ax &= b \\ L \cdot \underbrace{Ux}_y &= b \end{aligned}$$

$$\boxed{\begin{cases} Ly = b \\ Ux = y \end{cases}} \rightarrow \begin{cases} O(\frac{n^2}{2}) \\ O(\frac{n^2}{2}) \end{cases} = O(n^2)$$

Esecuzione:

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix}$$

Step 1: Eliminazione della prima colonna

Definiamo i coefficienti:

$$e_{i1} = -\frac{a_{i1}}{a_{11}}, \quad i = 2, \dots, n$$

e la matrice elementare:

$$E_1 = \begin{bmatrix} 1 & 0 & \dots & 0 \\ e_{21} & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ e_{n1} & 0 & \dots & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ -\frac{a_{21}}{a_{11}} & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ -\frac{a_{n1}}{a_{11}} & 0 & \dots & 1 \end{bmatrix}$$

Applichiamo E_1 a A per ottenere:

$$A_2 = E_1 A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ 0 & a_{22}^{(2)} & \dots & a_{2n}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & a_{n2}^{(2)} & \dots & a_{nn}^{(2)} \end{bmatrix}$$

—

Step k : Eliminazione della colonna k

In generale, al passo k definiamo:

$$e_{ik} = -\frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}, \quad i = k+1, \dots, n$$

e la matrice elementare:

$$E_k = \begin{bmatrix} I_{k-1} & 0 \\ 0 & \tilde{E}_k \end{bmatrix}, \quad \tilde{E}_k = \begin{bmatrix} 1 & 0 & \dots & 0 \\ e_{k+1,k} & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ e_{nk} & 0 & \dots & 1 \end{bmatrix}$$

Dove I_{k-1} è l'identità di dimensione $(k-1) \times (k-1)$. Applicando E_k alla matrice aggiornata A_k otteniamo:

$$A_{k+1} = E_k A_k = \begin{bmatrix} * & * & \dots & * \\ 0 & * & \dots & * \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & * \end{bmatrix}$$

Costo al passo k : $O((n-k)^2)$.

Costo totale $E_1, \dots, E_k$: $\sum_{i=1}^{n-1} O(i^2) \approx O(\frac{n^3}{3})$.

Ripetendo questo procedimento fino a $k = n - 1$, otteniamo la matrice triangolare superiore U :

$$U = E_{n-1} \dots E_1 A$$

Ogni matrice E_k e' invertibile e si puo' scrivere:

$$A = E_1^{-1} E_2^{-1} \dots E_{n-1}^{-1} U$$

Definiamo $L = E_1^{-1} E_2^{-1} \dots E_{n-1}^{-1}$

$$\rightarrow A = LU$$

con L triangolare inferiore con diagonale 1, U triangolare superiore.

Il costo computazionale della fattorizzazione LU e' $O(\frac{2n^3}{3}) + 2O(\frac{n^2}{2})$.

Tutte le matrici non singolari posso essere fattorizzate LU?

Prendiamo in considerazione

$$A = \begin{pmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{pmatrix}$$

Dobbiamo calcolare L :

$$L = \begin{pmatrix} 1 & 0 & 0 \\ x & 1 & 0 \\ x & x & 1 \end{pmatrix}$$

Quindi calcoliamo le matrici elementari per poi avere L , partiamo con E_1 :

$$e_{11} = -\frac{a_{11}}{a_{11}} = -\frac{0}{0} \\ \rightarrow \text{Impossibile}$$

$\rightarrow$ Non possiamo fattorizzare A con LU !

Modifica algoritmo LU: Permutazione

Idea: ad ogni passo k prima di calcolare E_k scambio le righe di A_k tale che

$$a_{kk} = \max_{i=k, \dots, n} \{(a_{i,k})\}$$

Esempio: riprendiamo in considerazione la matrice A di prima

$$A = \begin{pmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{pmatrix}$$

Definiamo P_1 la matrice di **permutazione** (derivante da I scambiando le righe):

$$P_1 = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}$$

Allora attraverso il prodotto

$$A_0 = P_1 A = \begin{pmatrix} 6 & 7 & 8 \\ 3 & 4 & 5 \\ 0 & 1 & 2 \end{pmatrix}$$

E ora applichiamo LU normalmente:

$$E_1 = \begin{pmatrix} x & x & x \\ 0 & x & x \\ 0 & x & x \end{pmatrix}$$

$$A_1 = E_1 A_0$$

E poi riappliciamo $a_{22} = \max_{i=2,\dots,n} \{(a_{i,2})\}$, e così' via.

Questo algoritmo che abbiamo applicato alla matrice A si chiama fattorizzazione

$$P \cdot A = L \cdot U$$

In soldoni:

$$Ax = b$$

$$PAx = Pb$$

$$LUx = Pb$$

$$\rightarrow \boxed{\begin{cases} Ly = Pb \\ Ux = y \end{cases}}$$

Definizione: Se A e' non singolare ($n \times n$) allora esistono:

- P matrice di permutazione
- L matrice triangolare inferiore con 1 sulla diagonale
- U matrice triangolare superiore

tali che

$$PA = LU$$

Questo algoritmo e' **stabile!** (errore alg. limitato)

5.3 Condizionamento

Teorema Rouché Capelli

Dato $A \in \mathbb{R}^{m \times n}, x \in \mathbb{R}^n, b \in \mathbb{R}^m$:

- Il sistema lineare $Ax = b$ ammette soluzione unica se $rk(A) = rk(A|b) = n$
- Il sistema $Ax = b$ ha infinite soluzioni se $rk(A) = rk(A|b) < n$
- Il sistema $Ax = b$ non ha soluzioni se $rk(A) \neq rk(A|b)$

Possiamo anche *simplificarlo* come

- Se $m = n$ e le equazioni sono indipendenti $\Rightarrow$ soluzione unica.
- Se $m < n$ (sottodeterminato) $\Rightarrow$ infinite soluzioni possibili se compatibile.
- Se $m > n$ (sovradeterminato) $\Rightarrow$ generalmente nessuna soluzione.

Dato un sistema di dati in input (A, b) e il rispettivo output $x = A^{-1}b$ se viene aggiunto un **errore sui dati** $(\Delta A, \Delta b)$:

$$\begin{aligned} (A, b) &\xrightarrow{A^{-1}} x \\ (A + \Delta A, b + \Delta b) &\xrightarrow{(A + \Delta A)^{-1}} x + \Delta x \end{aligned}$$

Ma se nel nostro modello predittivo otteniamo un errore risultato maggiore dell'errore sui dati previsto

$$\|\Delta x\| \gg \|\Delta A, \Delta b\|$$

abbiamo un modello inesatto.

Sia x^* soluzione del sistema $Ax = b$, cerchiamo la soluzione $\tilde{x}$ del sistema perturbato:

$$(A + \Delta A)\tilde{x} = b + \Delta b$$

Esempio:

$$A = \begin{pmatrix} 1 & 2 \\ 0.4999 & 1.001 \end{pmatrix} \quad (b + \Delta b) = \begin{pmatrix} 3 \\ 1.5 \end{pmatrix}$$

Soluzione reale: $\Delta A = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad \Delta b = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$

$$\begin{aligned} \begin{cases} 1x_1^* + 2x_2^* = 3 \\ 0.4999x_1^* + 1.001x_2^* = 1.5 \end{cases} \\ \rightarrow \begin{cases} x_1^* = 1 \\ x_2^* = 1 \end{cases} \end{aligned}$$

Soluzione del sistema perturbato: $\Delta A = \begin{pmatrix} 0.1 \\ 0.1 \end{pmatrix}$, $\Delta b = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$

$$\begin{cases} 1\tilde{x}_1 + 2\tilde{x}_2 = 3 \\ 0.5\tilde{x}_1 + 1.002\tilde{x}_2 = 1.5 \end{cases} \rightarrow \begin{cases} \tilde{x}_1 = 3 \\ \tilde{x}_2 = 0 \end{cases}$$

Notiamo come per un minusco incremento SOLO per ΔA il risultato sia completamente sbalato!

Infatti una piccola differenza nella matrice come ΔA

$$\frac{\|\Delta A\|}{\|A\|} \approx 5.7 \cdot 10^{-4}$$

Porta un grande cambiamento nella soluzione

$$\frac{\|\tilde{x} - x^*\|}{\|x^*\|} \approx 1.58$$

I problemi che soffrono di grandi perturbazioni nei risultati per colpa di piccole perturbazioni nei dati sono definiti **mal condizionati**.

5.4 Problema minimi quadrati e SVD

Possiamo risolvere $Ax = b$ attraverso il **metodo dei minimi quadrati**

$$\begin{aligned} \min_x \|Ax - b\|_2^2 \\ \Rightarrow f(x) = \|Ax - b\|_2^2 \\ \Rightarrow f(x) = (Ax - b)^T (Ax - b) \end{aligned}$$

Noi cerchiamo $x \mid \nabla f(x) = 0$:

$$\begin{aligned} \frac{d}{dx} [(Ax - b)^T (Ax - b)] &= 0 \\ \Rightarrow \boxed{A^T Ax = A^T b} \end{aligned}$$

Così avremo una matrice quadrata e definita positiva (SPD)!
Costo computazionale tramite equazioni normali: $O(mn^2 + n^3)$.

Potremmo anche risolverlo con **Cholesky** + LU: $O(mn^2 + \frac{1}{3}n^3)$

Una coppia di vettori $v, p \neq 0 \in \mathbb{R}^n$ si dice **ortogonale** se

$$v^T p = 0$$

Ovvero se il loro prodotto scalare è 0.

Una coppia di vettori v, p si dice **ortonormale** se sono ortogonali e di lunghezza unitaria 1:

$$\begin{aligned}v^T p &= 0 \\v^T v &= \|v\|^2 = 1 \\p^T p &= \|p\|^2 = 1\end{aligned}$$

Una matrice W ($n \times n$) si dice **ortogonale** se le sue colonne sono vettori ortonormali

$$W = \begin{pmatrix} 1/\sqrt{3} & 0 & 1/\sqrt{2} \\ 1/\sqrt{3} & 1 & 1/\sqrt{2} \\ 1/\sqrt{3} & 0 & 0 \end{pmatrix}$$

$$w_{ij} = \begin{cases} 0 & \text{se } i \neq j \\ 1 & \text{se } i = j \end{cases}$$

- $W^T = W^{-1}$
- $\forall x \in \mathbb{R}^n, \quad \|x\|_2 = \|Wx\|_2$ (isometrica)

Teorema di diagonalizzazione

Se gli autovettori di A sono x_i allora

$$A = PDP^{-1}$$

$$D = \begin{pmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \lambda_n \end{pmatrix}, \quad P = (x_1, \dots, x_n)$$

Data una matrice A $m \times n$ la SVD costruisce basi ortogonali di $\mathbb{R}^m$ e $\mathbb{R}^n$ che permettono di rappresentare A attraverso una matrice diagonale.

Teorema: Sia A matrice $m \times n$ con $k = rk(A)$ ($\leq n \leq m$), allora:

- U ($m \times m$) matrice ortogonale
- V ($n \times n$) matrice ortogonale
- Σ ($m \times n$) matrice diagonale

$$A = U\Sigma V^T$$

$$\Sigma = \begin{pmatrix} \sigma_1 & 0 & 0 & 0 & 0 \\ 0 & \sigma_2 & 0 & 0 & 0 \\ 0 & 0 & \sigma_k & 0 & 0 \\ 0 & 0 & 0 & \sigma_{k+1} & 0 \\ 0 & 0 & 0 & 0 & \sigma_n \end{pmatrix}$$

Nota:

- $\sigma_1 \geq \dots \geq \sigma_n \geq 0$ sono i **valori positivi**
- $\sigma_1 \geq \dots \geq \sigma_k > 0$
- $\sigma_{k+1} = \dots = \sigma_n = 0$
- Le colonne di U sono i vettori singolari sinistri
- Le colonne di V sono i vettori singolari destri

Nota: se $n > m$ possiamo sempre trovare la SVD anche se in questa Σ diventa $n \times m$.

m > n

$$\begin{matrix} n \\ \boxed{A} \\ m \end{matrix} = \begin{matrix} m \\ \boxed{U} \\ m \end{matrix} \begin{matrix} n \\ \boxed{\Sigma} \\ m \end{matrix} \begin{matrix} n \\ \boxed{V^T} \\ n \end{matrix} \quad (1)$$

m ≤ n

$$\begin{matrix} m \\ \boxed{A} \\ m \end{matrix} = \begin{matrix} m \\ \boxed{U} \\ m \end{matrix} \begin{matrix} m \\ \boxed{\Sigma} \\ m \end{matrix} \begin{matrix} n \\ \boxed{V^T} \\ m \end{matrix} \quad (2)$$

Teorema: Sia A la matrice $(m \times n)$ con $k = rk(A) (\leq n \leq m)$ allora possiamo scrivere

$$A = \sum_{i=1}^k \sigma_i u_i v_i^T$$

Ovvero somma di matrici di rango 1.

Gli autovalori λ_i si possono calcolare per $A \in \mathbb{R}^{n \times n}$ con

$$\det(A - \lambda I) = 0$$

Pero' questo si calcola solo se A e' matrice quadrata!
Abbiamo sempre n autovalori (inclusi i nulli).

Ricorda: $rk(A) = rk(A^T A)$.

Abbiamo la seguente relazione tra gli autovalori di $A^T A$ e i valori singolari di A :

$$\begin{aligned}
 A^T A &= (U \Sigma V^T)^T (U \Sigma V^T) \\
 &= (V \Sigma^T U^T) (U \Sigma V^T) \\
 &= V \Sigma^T \Sigma V^T \\
 &= V \Sigma^2 V^T \\
 &= V \begin{pmatrix} \sigma_1^2 & 0 & 0 \\ 0 & \sigma_2^2 & 0 \\ 0 & 0 & \sigma_n^2 \end{pmatrix} V^T \\
 \Rightarrow \boxed{\sigma_j = \sqrt{\lambda_i \text{ di } (A^T A)}}
 \end{aligned}$$

$\Rightarrow$ abbiamo $rk(A)$ autovalori NON nulli e $n - rk(A)$ autovalori nulli!

Nota: gli autovalori λ_i di $A^T A$ possono essere in qualsiasi ordine; non c'è corrispondenza diretta tra l'indice i di λ_i e l'indice i di σ_i .

- $\sigma_1 = \sqrt{\lambda_{\max \text{ di } A^T A}} = \|A\|_2$
- $\sigma_n = \sqrt{\lambda_{\min \text{ di } A^T A}} = \|A^{-1}\|_2 \rightarrow \|A^{-1}\| = \frac{1}{\sigma_n}$
- $cond(A) = \frac{\sigma_1}{\sigma_n}$

Prendiamo ora in considerazione $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $x \in \mathbb{R}^n$

$$Ax = b$$

Per Rouchè Capelli sappiamo che non ha soluzioni quindi

$$Ax - b \neq 0$$

definiamo r il vettore **residuo**

$$r = Ax - b$$

Quindi più se A sono i dati in input, b output desiderato e x quello che abbiamo imparato possiamo definire r come errore del modello.

Nel capitolo 3 avevamo definito il problema dei minimi quadrati (lineare) come:

$$\min_{\theta \in \mathbb{R}^k} \|\Phi \theta - y\|_2^2$$

che nel caso del errore residuo r

$$r = \min_x \|Ax - b\|_2^2$$

Questo è un problema **Linear Least Square**.

Come calcolare la soluzione? Sapendo che r è una funzione convessa:

- $rk(A) = n = m \Rightarrow$ 1 sola soluzione unica $\Rightarrow$ gradiente (iterativo), equazioni normali / Cholesky + LU oppure SVD diretta.

- $n \neq m$ e $b \notin \text{Im}(A)$:
 - nessuna soluzione esatta $\Rightarrow$ Trovare $\min_x \|Ax - b\|_2$: gradiente (iterativo) oppure SVD-pseudoinversa.
- $n \neq m$ e $b \in \text{Im}(A)$:
 - Sovradeterminato ($m > n$): soluzione unica $\Rightarrow$ gradiente (iterativo), equazioni normali / Cholesky + LU, oppure SVD-pseudoinversa.
 - Sottodeterminato ($m < n$): infinite soluzioni compatibili $\Rightarrow$ gradiente (converge alla soluzione a norma euclidea minima) oppure SVD-pseudoinversa (restituisce soluzione norma euclidea minima).

Risoluzione di un problema LSQ con SVD

Forma sommatoria: Abbiamo visto che possiamo trovare la soluzione per $\text{rk}(A) = n = m$

$$\begin{aligned} Ax &= b \\ \Rightarrow U\Sigma V^T \hat{x}^* &= b \\ \Rightarrow \boxed{\hat{x}^* = V\Sigma^{-1}U^T b} \end{aligned}$$

Costo computazionale: $O(\frac{4}{3}mn^2)$.

Ma se $m \neq n$ o anche se $\text{rk}(A) < n = m$ avremo dei $\sigma_{k+1} = 0$ che implica che $\nexists \Sigma^{-1}$.

Quindi come facciamo?

Teorema pseudoinversa: la soluzione $\hat{x}^*$ e' data ($r = \text{rk}(A) = n$):

$$\hat{x}^* = A^+ b = V\Sigma^+ U^T b = \sum_{i=1}^r \frac{u_i^T b}{\sigma_i} v_i$$

$$\|\hat{x}^*\|_2^2 = \sum_{i=1}^r \left(\frac{u_i^T b}{\sigma_i} v_i\right)^2 \quad \|r^*\| = \sum_{i=r+1}^n (u_i^T b)^2$$

Sia $A \in \mathbb{R}^{m \times n}$ con $\text{rk}(A) \leq \min(m, n)$ e $A = U\Sigma V^T$. Si definisce **pseudoinversa**:

$$\begin{aligned} A^+ &= V\Sigma^+ U^T \\ (\Sigma^+)_{ij} &= \begin{cases} \frac{1}{\sigma_i} & \text{se } i = j \text{ e } i \leq k \\ 0 & \text{altrimenti} \end{cases} = \begin{pmatrix} \frac{1}{\sigma_1} & 0 & 0 \\ 0 & \frac{1}{\sigma_2} & 0 \\ 0 & 0 & \frac{1}{\sigma_k} \end{pmatrix} \end{aligned}$$

Se $\text{rk}(A) = m = n \rightarrow A^+ = A^{-1} \Rightarrow \Sigma^+ = \Sigma^{-1}$

Costo computazionale: $O(\frac{4}{3}mn^2)$

Definizione: il numero di condizione in norma 2 di matrice $m \times n$ con valori singolari ($\sigma_1 > \dots > \sigma_k > \sigma_{k+1} = \dots = \sigma_n = 0$):

$$\text{cond}(A) = \frac{\sigma_1}{\sigma_k}$$

- $cond(A) \approx 1$ matrice ben condizionata
 - Piccoli errori nei dati $\rightarrow$ piccoli errori nel risultato
- $cond(A) \gg 1$ matrice mal condizionata
 - Piccoli errori nei dati $\rightarrow$ grandi errori nel risultato
 - Numericamente rischioso risolvere sistemi lineari $Ax = b$

Possiamo utilizzare come indici per la precisione

- Errore residuo: $Xa - y$
- MSE (mean square error): $media((x^* - x)^2)$
- R^2 (Coefficiente di determinazione), indica quanto il modello spiega la varianza dei dati:

$$1 - \frac{\sum_i (x_i^* - x_i)^2}{\sum_i (x_i - \bar{x})^2}$$

6 Imaging e problemi inversi

Data una immagine in scala di grigi possiamo definire la matrice A come

$$A[i][j] = \text{intensita' del pixel}$$

6.1 Compressione SVD

Possiamo rendere piu' efficiente una immagine attraverso la compressione:

$$A \approx A_p = \sum_{i=1}^p u_i \cdot \sigma_i \cdot v_i^T$$

con $p \leq rk(A)$.

Questo funziona perche σ_k contiene piu' informazioni di σ_{k+1} . Quindi prendendo solo i primi p valori singolari possiamo avere quasi tutte le informazioni pero' memorizziamo meno informazioni.

A livello teorico le due matrici contengono lo stesso numero di informazioni (NON GLI STES-
SI!!!), quindi salvandoli su disco il peso non dovrebbe variare. Pero' nei formati come *png*, *jpeg*,
ecc. ci sono delle compressioni interne gia' integrate e quindi grazie alla "*pulizia*" del SVD
otteniamo delle buone compressioni!

Definiamo errore relativo

$$E_r = \frac{\|A - A_p\|_2}{\|A\|_2}$$

Definiamo fattore di compressione

$$c_p = \frac{1}{p} \min(m, n)$$

6.2 Deblur

Definiamo x come l'immagine sorgente, possiamo dire che la funzione di blur (Point Spread Function (PSF)) e':

$$A(x)$$

ovvero

$$A(x) = [K * x \mid P]$$

con K un kernel di convoluzione e P il suo pattern di estensione.

Ora definiamo w come vettore di rumore (gaussiano), possiamo definire

$$y^\delta = A(x) + w = [K * x \mid P] + w$$

Quindi y^δ e' l'immagine con blur e rumore.

Come otteniamo l'immagine originaria? Abbiamo A come funzione di blur e y^δ come immagine distorta.

$$\min_x \|Ax - y^\delta\|_2^2$$

- Metodo dei minimi quadrati: $A^T Ax = A^T y^\delta \Rightarrow O(mn^2 + n^3)$
- SVD: $O(\frac{4}{3}mn^2)$
- Metodo dei minimi quadrati con **CGLS**: $\approx O(k \cdot mn)$ con $k \in \mathbb{R}$

Definiamo l'errore relativo

$$E_r = \frac{\|x_{\text{originale}} - x\|_2^2}{\|x_{\text{originale}}\|_2^2}$$

Regolarizzazione di Tikhonov

Possiamo ulteriormente migliorare il deblur attraverso la regolarizzazione del rumore. Partiamo da $y^\delta = Ax + w$, definiamo la regolarizzazione

$$x_\lambda = \arg \min_x \|Ax - y^\delta\|_2^2 + \lambda^2 \|x\|_2^2$$

Dove

- $\|Ax - y^\delta\|_2^2$ indica la fedeltà sulla immagine originale (fit dei dati)
- $\lambda^2 \|x\|_2^2$ indica la complessità della soluzione, ovvero quanto dobbiamo limitare la "rumorosità" della soluzione (regolarizzazione)

Questo è **Tikhonov 1-ordine**, serve quando abbiamo $A(x)$ mal condizionata quindi con piccole variazioni abbiamo grandi sbalzi, con il 1 - *ordine* possiamo regolarizzare e ottenere una soluzione stabile con picchi rumorosi ridotti.

Dobbiamo risolvere la funzione

$$\begin{aligned} \Rightarrow f(x) &= \|Ax - y^\delta\|_2^2 + \lambda^2 \|x\|_2^2 \\ \Rightarrow f(x) &= (Ax - y^\delta)^T (Ax - y^\delta) + (\lambda^2 \|x\|_2^2)^T (\lambda^2 \|x\|_2^2) \end{aligned}$$

e dato che minimizziamo cerchiamo la x minima ovvero $x|\nabla f(x) = 0$:

$$\begin{aligned} \frac{d}{dx} [(Ax - y^\delta)^T (Ax - y^\delta) + (\lambda^2 \|x\|_2^2)^T (\lambda^2 \|x\|_2^2)] &= 0 \\ \Rightarrow (A^T A + \lambda^2)x &= A^T y^\delta \end{aligned}$$

Ora possiamo scegliere il nostro metodo preferito per risolvere la SPD.

Ma come scegliamo il parametro di regolarizzazione λ ?

Teoricamente possiamo definirlo come

$$x_{opt} = \min_{\lambda} \|x_\lambda - x_{GT}\|_2$$

Questo vuol dire minimizzare l'errore assoluto, sarebbe la cosa migliore da fare ma implica conoscere x_{GT} che nella realtà è impossibile.

E come lo calcoliamo senza sapere x_{GT} ? Un metodo è il **principio di massima discrepanza**:

$$\begin{aligned} \|Ax_\lambda - y^\delta\|_2 &\approx \|w\|_2 \\ \Rightarrow \lambda_{DP} &= \arg \min_{\lambda} |\|A(x_\lambda) - y^\delta\|_2 - \|w\|_2| \end{aligned}$$

Generalmente siamo su $\lambda_{DP} \approx 0.01$.

Questo però richiede che conosciamo il rumore w , però normalmente non è noto.

Per la valutazione del risultato ottenuto facciamo

$$E_r = \frac{\|x_{GT} - x_{tikhonov}\|_2^2}{\|x_{GT}\|_2^2}$$

Regolarizzazione con variazione totale

Un altro metodo di regolarizzazione molto utilizzato e'

$$TV(x) = \|\nabla x\|_1$$

Ma dato che non e' differenziabile in (0,0) aggiungiamo un piccolo β :

$$TV_\beta(x) = \sum_{i,j} \sqrt{(\nabla x_{i,j})^2 + \beta^2}$$

Quindi dobbiamo risolvere

$$\min_x \|Ax - y^\delta\|_2^2 + \lambda TV_\beta(x)$$

Il valore λ come lo scegliamo? A valore fisso, oppure con metodi iterativi (anche DP). Questo puo' essere risolto con il metodo del gradiente + backtracking.

Algorithm 3 TV + Discrepancy Principle con risoluzione via GD + regolarizzazione

```
1:  $\lambda_{\text{list}} = [\dots]$ 
2:  $\text{soluzioni}[]$ 
3:  $\text{residui}[]$ 
4: for  $\lambda$  in  $\lambda_{\text{list}}$  do
5:    $x, \text{alpha}, \text{storico\_iterazioni}, n\text{-iter} = \text{GD\_REG}(A, y^\delta, n = 1.0, p = 1, \text{reg} =$ 
      $\lambda, TV()_\beta, q = 1)$ 
6:    $\text{soluzioni.append}(x)$ 
7:    $\text{residui.append}(\|Ax - y^\delta\|_2^2)$ 
8: end for
9:  $idx_{\text{DP}} = \text{index}(\min(\text{residui} - w), \text{residui})$ 
10:  $\lambda_{\text{DP}} = \lambda_{\text{list}}[idx_{\text{DP}}]$ 
11:  $x_{\text{final}} = \text{soluzioni}[idx_{\text{DP}}]$ 
```
